

Contents

1	Table of Contents	5
1.1	Overview of Digital Design with Verilog HDL	5
1.2	Hierarchical Modeling Concepts	5
1.3	Basic Concepts	5
1.4	Modules & Ports	5
1.5	Gate-Level Modeling	5
1.6	Dataflow Modeling	5
1.7	Behavioral Modeling	5
1.8	Tasks & Functions	5
1.9	Useful Modeling Techniques	5
2	Overview of Digital Design with Verilog HDL	6
2.1	1. Evolution of Digital Design and EDA	6
2.2	2. Emergence of Hardware Description Languages (HDLs)	6
2.3	3. Typical VLSI Design Flow	7
2.4	4. Importance of HDLs	7
2.5	5. Popularity of Verilog HDL	8
2.6	6. Trends in HDLs and Digital Design	8
2.6.1	Increasing Abstraction	8
2.6.2	Dominance of RTL Design	8
2.6.3	Advances in Verification	8
2.6.4	High-Performance Design Considerations	9
2.6.5	System-Level Design Methodology	9
3	Hierarchical Modeling Concepts	9
3.1	1. Design Methodologies	10
3.1.1	Top-Down Design Methodology	10
3.1.2	Bottom-Up Design Methodology	10
3.1.3	Combined Approach	10
3.2	2. Modules in Verilog	11
3.3	3. Levels of Abstraction in Verilog	11
3.3.1	Behavioral Level	11
3.3.2	Dataflow Level	11
3.3.3	Gate Level	11
3.3.4	Switch Level	12
3.3.5	Key Observations	12
3.4	4. Module Instantiation and Instances	12
3.4.1	Important Rules	12
3.5	7. Components of a Simulation	12
3.5.1	Design Block	12
3.5.2	Stimulus Block (Testbench)	12
3.5.3	Methods of Applying Stimulus	13
4	Verilog Basic Concepts	15
4.1	1. Overview	15
4.2	2. Lexical Conventions	15

4.2.1	General Rules	15
4.2.2	Whitespace	15
4.2.3	Comments	15
4.2.4	Operators	15
4.2.5	Number Representation	15
4.2.6	Strings	16
4.2.7	Identifiers and Keywords	16
4.2.8	Escaped Identifiers	16
4.3	3. Data Types in Verilog	16
4.3.1	Value Set	16
4.4	4. Nets	17
4.5	5. Registers	17
4.6	6. Vectors	17
4.6.1	Vector Operations	17
4.7	7. Integer, Real, and Time Data Types	17
4.7.1	Integer	17
4.7.2	Real	17
4.7.3	Time	18
4.8	8. Arrays	18
4.9	9. Memories	18
4.10	10. Parameters and Constants	18
4.10.1	Local Parameters	18
4.11	11. Strings in Registers	18
4.12	12. System Tasks	19
4.12.1	Displaying Information	19
4.12.2	Monitoring Signals	19
4.12.3	Simulation Control	19
4.13	13. Compiler Directives	19
4.13.1	<code>define</code>	19
4.13.2	<code>include</code>	19
5	Verilog Modules and Ports	20
5.1	1. Overview of Modules	20
5.1.1	Structure of a Module	20
5.2	2. Ports and Module Interface	20
5.2.1	Key Characteristics	22
5.3	3. Port List	22
5.4	4. Port Declarations	22
5.4.1	Important Rules	22
5.4.2	ANSI C Style Declaration	22
5.5	5. Port Connection Rules	22
5.5.1	Inputs	22
5.5.2	Outputs	23
5.5.3	Inouts	23
5.5.4	Width Matching	23
5.5.5	Unconnected Ports	23
5.6	6. Port Connection Methods	23
5.6.1	Ordered List Connection	23

5.6.2	Named Connection	23
5.7	7. Hierarchical Name Referencing	23
5.7.1	Key Concepts	23
5.7.2	Benefits	24
5.7.3	Special Usage	24
6	Gate-Level Modeling in Verilog	24
6.1	1. Overview of Gate-Level Modeling	24
6.2	2. Gate Primitives in Verilog	25
6.2.1	Categories of Gates	25
6.3	3. Gate Instantiation	26
6.4	4. Arrays of Gate Instances	26
6.5	5. Gate Delays	27
6.5.1	Types of Delays	27
6.6	6. Delay Specification	27
6.7	7. Min, Typical, and Max Delays	27
6.7.1	Usage	28
6.8	8. Timing Behavior and Simulation	28
7	Dataflow Modeling in Verilog	29
7.1	1. Overview of Dataflow Modeling	29
7.2	2. Continuous Assignments	29
7.2.1	Key Characteristics	29
7.3	3. Implicit Continuous Assignment	30
7.3.1	Key Point	30
7.4	4. Implicit Net Declaration	30
7.5	5. Delays in Continuous Assignments	30
7.5.1	5.1 Regular Assignment Delay	30
7.5.2	5.2 Implicit Assignment Delay	30
7.5.3	5.3 Net Declaration Delay	31
7.5.4	Key Insight	31
7.6	6. Expressions, Operands, and Operators	31
7.6.1	Expressions	31
7.6.2	Operands	31
7.6.3	Operators	31
7.7	7. Operator Types in Verilog	31
7.7.1	7.1 Arithmetic Operators	31
7.7.2	7.2 Logical Operators	32
7.7.3	7.3 Relational Operators	32
7.7.4	7.4 Equality Operators	32
7.7.5	7.5 Bitwise Operators	32
7.7.6	7.6 Reduction Operators	33
7.7.7	7.7 Shift Operators	33
7.7.8	7.8 Concatenation Operator	33
7.7.9	7.9 Replication Operator	34
7.7.10	7.10 Conditional Operator	34
7.8	8. Operator Precedence	34

8	Behavioral Modeling in Verilog	35
8.1	1. Overview of Behavioral Modeling	35
8.2	2. Structured Procedures: initial and always	36
	8.2.1 initial Block	36
	8.2.2 always Block	36
	8.2.3 Key Difference	36
8.3	3. Procedural Assignments	36
	8.3.1 3.1 Blocking Assignments (=)	36
	8.3.2 3.2 Nonblocking Assignments (<=)	37
	8.3.3 Key Insight	37
8.4	4. Timing Controls	37
	8.4.1 4.1 Delay-Based Timing Control	37
	8.4.2 4.2 Event-Based Timing Control	37
	8.4.3 4.3 Level-Sensitive Timing Control	38
8.5	5. Conditional Statements	38
	8.5.1 Types	38
	8.5.2 Key Behavior	39
8.6	6. Case Statements (Multiway Branching)	39
	8.6.1 Basic Case	39
	8.6.2 Variants	39
8.7	7. Looping Constructs	39
	8.7.1 7.1 While Loop	40
	8.7.2 7.2 For Loop	40
	8.7.3 7.3 Repeat Loop	40
	8.7.4 7.4 Forever Loop	40
8.8	8. Sequential vs Parallel Blocks	40
	8.8.1 Sequential Block (begin-end)	40
	8.8.2 Parallel Block (fork-join)	40
	8.8.3 Key Insight	41
8.9	9. Named Blocks and Control	41
	8.9.1 Named Blocks	41
	8.9.2 Disable Statement	41
8.10	10. Generate Blocks (Elaboration-Time Constructs)	41
	8.10.1 Key Characteristics	41
	8.10.2 10.1 Generate Loop	41
	8.10.3 10.2 Generate Conditional	42
	8.10.4 10.3 Generate Case	42
9	Tasks and Functions in Verilog	42
9.1	1. Overview	42
9.2	2. Key Differences Between Tasks and Functions	43
	9.2.1 Functional Distinction	43
	9.2.2 Summary of Differences	43
9.3	3. Tasks	43
	9.3.1 3.1 When to Use Tasks	43
	9.3.2 3.2 Task Declaration and Invocation	43
	9.3.3 Key Points	44
	9.3.4 3.3 Task with Timing Control	44

9.3.5	3.4 Automatic (Re-entrant) Tasks	44
9.3.6	Key Insight	44
9.4	4. Functions	45
9.4.1	4.1 When to Use Functions	45
9.4.2	4.2 Function Declaration and Return Mechanism	45
9.4.3	Key Insight	45
9.4.4	4.3 Function with Explicit Width	45
9.4.5	4.4 Automatic (Recursive) Functions	46
9.4.6	Key Insight	46
9.5	5. Constant Functions	46
9.6	6. Signed Functions	46
9.7	7. Best Practices	47

These are the notes of the book “Verilog HDL by Samir Palnitkar” written/compiled by Maitreya Ranade.

1 Table of Contents

1.1 Overview of Digital Design with Verilog HDL

1.2 Hierarchical Modeling Concepts

1.3 Basic Concepts

1.4 Modules & Ports

1.5 Gate-Level Modeling

1.6 Dataflow Modeling

1.7 Behavioral Modeling

1.8 Tasks & Functions

1.9 Useful Modeling Techniques

-
- Overview of Digital Design with Verilog HDL
 - 1. Evolution of Digital Design and EDA
 - 2. Emergence of Hardware Description Languages (HDLs)
 - 3. Typical VLSI Design Flow
 - 4. Importance of HDLs
 - 5. Popularity of Verilog HDL
 - 6. Trends in HDLs and Digital Design
 - * Increasing Abstraction
 - * Dominance of RTL Design
 - * Advances in Verification
 - * High-Performance Design Considerations
 - * System-Level Design Methodology
-

2 Overview of Digital Design with Verilog HDL

2.1 1. Evolution of Digital Design and EDA

Digital circuit design has evolved significantly from the use of vacuum tubes to transistors and eventually to integrated circuits (ICs). Early ICs were categorized based on scale:

- Small Scale Integration (SSI) contained a very small number of gates.
- Medium Scale Integration (MSI) increased capacity to hundreds of gates.
- Large Scale Integration (LSI) enabled thousands of gates per chip.
- Very Large Scale Integration (VLSI) allowed more than 100,000 transistors on a single chip.

As integration levels increased, manual design methods became impractical due to complexity. This led to the development of **Electronic Design Automation (EDA)** tools.

EDA combines: - CAD (Computer-Aided Design): Physical design tasks such as layout, placement, and routing. - CAE (Computer-Aided Engineering): Front-end processes such as simulation, synthesis, and timing analysis.

With VLSI, physical prototyping using breadboards became infeasible. Designers increasingly relied on simulation tools to verify functionality before fabrication. Logic simulation became a critical step to identify and fix functional issues early in the design cycle.

2.2 2. Emergence of Hardware Description Languages (HDLs)

Traditional programming languages such as C, Pascal, and FORTRAN are sequential in nature and unsuitable for modeling hardware, which operates concurrently.

Hardware Description Languages (HDLs) were introduced to model parallelism in digital systems. The most prominent HDLs are:

- Verilog HDL (introduced in 1983 by Gateway Design Automation)
- VHDL (developed under DARPA funding)

Initially, HDLs were used mainly for simulation and verification. Designers still had to manually convert HDL descriptions into gate-level schematics.

The introduction of logic synthesis in the late 1980s transformed this workflow. Designers could describe circuits at the Register Transfer Level (RTL), specifying data flow and behavior. Logic synthesis tools automatically translated RTL descriptions into gate-level implementations.

This shift enabled designers to focus on functionality rather than low-level gate connections. HDLs also became widely used for system-level design, including modeling of FPGAs, buses, and complete systems.

Verilog HDL was standardized as IEEE 1364-1995 and later updated in 2001.

2.3 3. Typical VLSI Design Flow

The modern VLSI design flow using HDLs consists of the following stages:

1. Specification
The design process begins with defining the functional requirements, architecture, and interfaces of the system. At this stage, implementation details are not considered.
2. Behavioral Description
A high-level model of the system is created to evaluate functionality, performance, and compliance. This description is often written using HDLs.
3. RTL Design
The behavioral model is refined into a Register Transfer Level (RTL) description. This stage defines how data moves between registers and how operations are performed.
4. Logic Synthesis
RTL descriptions are converted into a gate-level netlist. The synthesis process ensures that the design meets constraints related to timing, area, and power.
5. Place and Route
The gate-level netlist is mapped onto a physical layout. This includes placement of components and routing of interconnections.
6. Verification and Fabrication
The physical design is verified before being sent for fabrication.

Most design effort is concentrated at the RTL level, where optimization has the greatest impact. RTL-based design significantly reduces development time from years to months and allows multiple design iterations.

Behavioral synthesis tools, which convert high-level descriptions directly into RTL, are emerging but not yet widely adopted.

It is important to note that EDA tools are only as effective as their inputs. Poorly written RTL can result in inefficient designs, following the principle of “Garbage In, Garbage Out (GIGO)”.

2.4 4. Importance of HDLs

HDLs provide several advantages over traditional schematic-based design:

- Technology Independence
Designers can write RTL without targeting a specific fabrication technology. The same design can be synthesized for different technologies with appropriate optimization.
- Early Functional Verification
Bugs can be identified and resolved at the RTL stage, reducing the likelihood of costly errors in later stages.
- Higher Level of Abstraction
Designers can focus on system behavior and data flow rather than low-level gate implementation.

- Improved Readability and Maintainability
Text-based descriptions with comments are easier to understand and debug compared to complex schematics.

HDLs are now the standard approach for designing complex digital systems and are indispensable for modern digital designers.

2.5 5. Popularity of Verilog HDL

Verilog HDL has become one of the most widely used hardware description languages due to the following features:

- It has a syntax similar to the C programming language, making it easy to learn for software engineers.
- It supports multiple levels of abstraction, including behavioral, RTL, gate-level, and switch-level modeling.
- It is supported by most logic synthesis and simulation tools.
- It is widely supported by semiconductor vendors through standard libraries.
- It includes the Programming Language Interface (PLI), which allows integration with C code for customization and advanced functionality.

These advantages have made Verilog a preferred choice for both design and verification.

2.6 6. Trends in HDLs and Digital Design

2.6.1 Increasing Abstraction

As circuit complexity continues to grow, designers are working at higher levels of abstraction. They focus on functionality, while EDA tools handle implementation details.

2.6.2 Dominance of RTL Design

RTL-based design remains the most widely used methodology. Logic synthesis tools efficiently convert RTL into gate-level netlists.

Behavioral synthesis, which operates at a higher level of abstraction, exists but has not achieved widespread adoption.

2.6.3 Advances in Verification

New verification techniques have emerged to improve design correctness:

- Formal Verification
Uses mathematical methods to prove correctness and equivalence between RTL and gate-level designs.
- Assertion-Based Verification
Embeds checks within RTL code to validate critical conditions during simulation.

- Advanced Verification Languages
Combine hardware modeling with object-oriented programming concepts to enable automated testing and coverage analysis. These languages complement, rather than replace, HDLs.

2.6.4 High-Performance Design Considerations

For timing-critical designs such as microprocessors, purely RTL-based synthesis may not yield optimal results. Designers often incorporate gate-level elements into RTL descriptions to achieve better timing performance.

2.6.5 System-Level Design Methodology

A mixed bottom-up approach is commonly used in large systems. Designers combine:

- Custom RTL modules
- Behavioral models
- Vendor-provided IP cores

This approach enables faster system integration, early simulation, and reduced development costs.

-
- Hierarchical Modeling Concepts
 - 1. Design Methodologies
 - * Top-Down Design Methodology
 - * Bottom-Up Design Methodology
 - * Combined Approach
 - 2. Modules in Verilog
 - 3. Levels of Abstraction in Verilog
 - * Behavioral Level
 - * Dataflow Level
 - * Gate Level
 - * Switch Level
 - * Key Observations
 - 4. Module Instantiation and Instances
 - * Important Rules
 - 7. Components of a Simulation
 - * Design Block
 - * Stimulus Block (Testbench)
 - * Methods of Applying Stimulus
 - Method 1: Stimulus as Top-Level Module
 - Method 2: Separate Top-Level Module
-

3 Hierarchical Modeling Concepts

- Hierarchical modeling is a fundamental concept in digital design that enables designers to manage complexity by structuring designs into smaller, reusable components.
- A well-defined design methodology is essential for efficient HDL-based design.

- A digital system is composed of multiple interconnected components. Understanding how these components are structured and interact is critical for both design and simulation.
-

3.1 1. Design Methodologies

Digital design primarily follows two methodologies: top-down and bottom-up. In practice, a combination of both is used.

3.1.1 Top-Down Design Methodology

- The design process begins by defining the top-level system functionality.
- The system is progressively decomposed into smaller sub-blocks.
- This decomposition continues until indivisible units, called leaf cells, are reached.
- This approach emphasizes system-level planning and architecture.

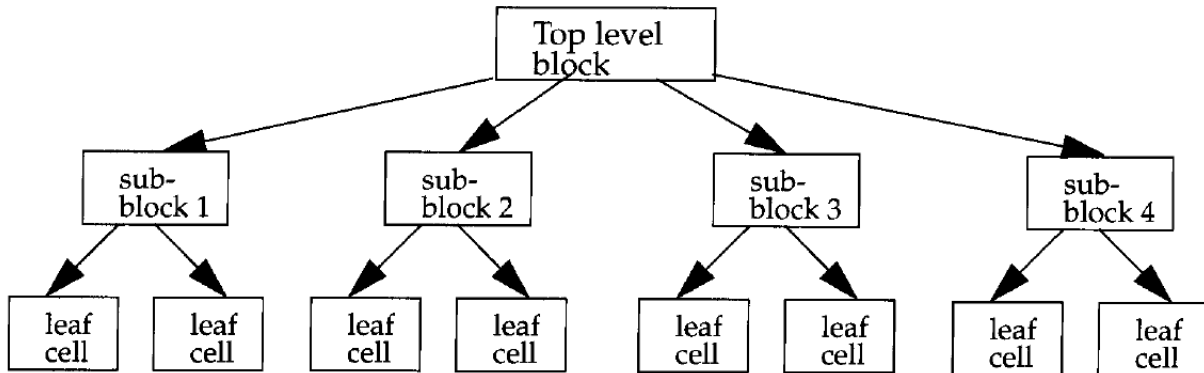


Figure 1: Top-Down Design Methodology

3.1.2 Bottom-Up Design Methodology

- The design begins with basic building blocks such as logic gates or primitive cells.
- These blocks are combined to form higher-level modules.
- The process continues until the complete system is constructed.
- This approach focuses on optimized implementation of low-level components.

3.1.3 Combined Approach

- In real-world designs, both methodologies are used together.
 - System architects define the overall structure using a top-down approach.
 - Circuit designers simultaneously develop optimized leaf cells using a bottom-up approach.
 - The two approaches converge at an intermediate level, typically at standard building blocks such as flip-flops.
-

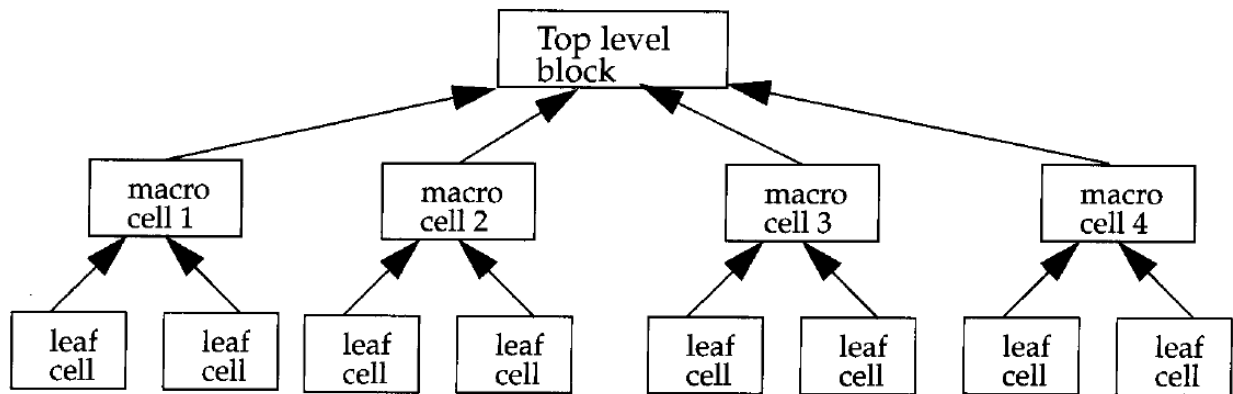


Figure 2: Bottom-Up Design Methodology

3.2 2. Modules in Verilog

A module is the fundamental building block in Verilog.

- A module can represent either a simple element or a complex system composed of submodules.
- It provides functionality through a defined interface consisting of input and output ports.
- The internal implementation of a module is hidden from other modules, enabling abstraction and modularity.
- This encapsulation allows internal changes without affecting the rest of the design.

A module is defined using the module and endmodule keywords and includes: - A unique module name - A list of input and output ports - Internal implementation

3.3 3. Levels of Abstraction in Verilog

Verilog supports multiple levels of abstraction to describe the same hardware module. The external behavior of the module remains consistent regardless of abstraction level.

3.3.1 Behavioral Level

- This is the highest level of abstraction.
- The design is described using algorithms without specifying hardware details.
- It is similar to high-level programming languages like C.

3.3.2 Dataflow Level

- The design is expressed in terms of data movement between registers.
- The designer specifies how data is processed and transferred.

3.3.3 Gate Level

- The design is described using logic gates and their interconnections.
- This corresponds closely to schematic-based design.

3.3.4 Switch Level

- This is the lowest level of abstraction.
- The design is described using switches and storage nodes.
- It requires detailed knowledge of transistor-level behavior.

3.3.5 Key Observations

- Designers can mix different abstraction levels within the same design.
 - RTL (Register Transfer Level) typically combines behavioral and dataflow modeling.
 - Higher abstraction levels provide better flexibility and portability.
 - Lower abstraction levels offer more control but reduce flexibility and increase complexity.
-

3.4 4. Module Instantiation and Instances

A module acts as a template from which instances are created.

- Instantiation is the process of creating a specific instance of a module.
- Each instance is a unique object with its own name, signals, and parameters.
- Multiple instances of the same module can exist in a design.

For example, a ripple carry counter may instantiate multiple T flip-flop modules, each with unique connections.

3.4.1 Important Rules

- Each instance must have a unique name.
- Module definitions cannot be nested inside other module definitions.
- Modules must be instantiated to be used; defining a module alone does not include it in the design.

This distinction between module definition and module instance is critical in Verilog design.

3.5 7. Components of a Simulation

After designing a module, its functionality must be verified through simulation.

3.5.1 Design Block

- Represents the actual hardware design to be tested.

3.5.2 Stimulus Block (Testbench)

- Provides input signals to the design block.
- Observes and verifies output responses.
- Typically written in Verilog.

It is considered good practice to keep the design and stimulus blocks separate to improve modularity and reusability.

3.5.3 Methods of Applying Stimulus

3.5.3.1 Method 1: Stimulus as Top-Level Module

- The stimulus block instantiates the design block.
- It directly drives input signals and monitors outputs.
- The stimulus block acts as the top-level module.

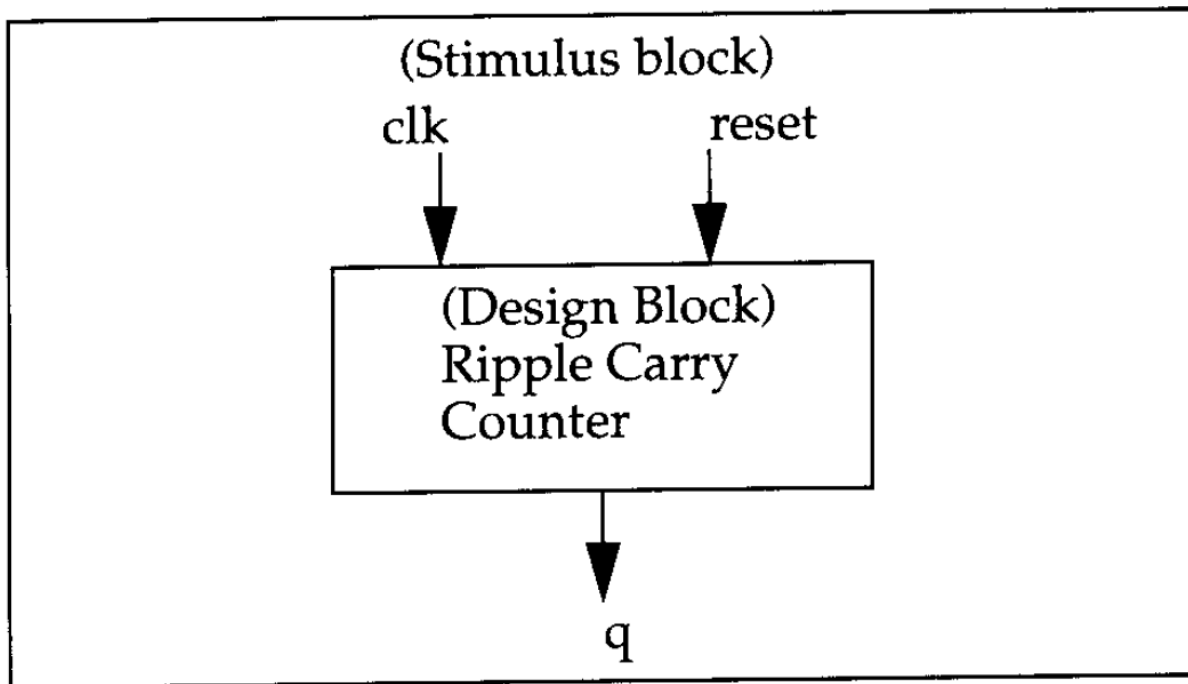


Figure 3: Stimulus as Top-Level Module

3.5.3.2 Method 2: Separate Top-Level Module

- A dummy top-level module instantiates both the design and stimulus blocks.
- The stimulus interacts with the design through defined interfaces.
- This approach improves modularity and separation of concerns.

Both methods are valid and can be chosen based on design and testing requirements.

-
- Verilog Basic Concepts
 - 1. Overview
 - 2. Lexical Conventions
 - * General Rules
 - * Whitespace
 - * Comments
 - * Operators
 - * Number Representation
 - Sized Numbers

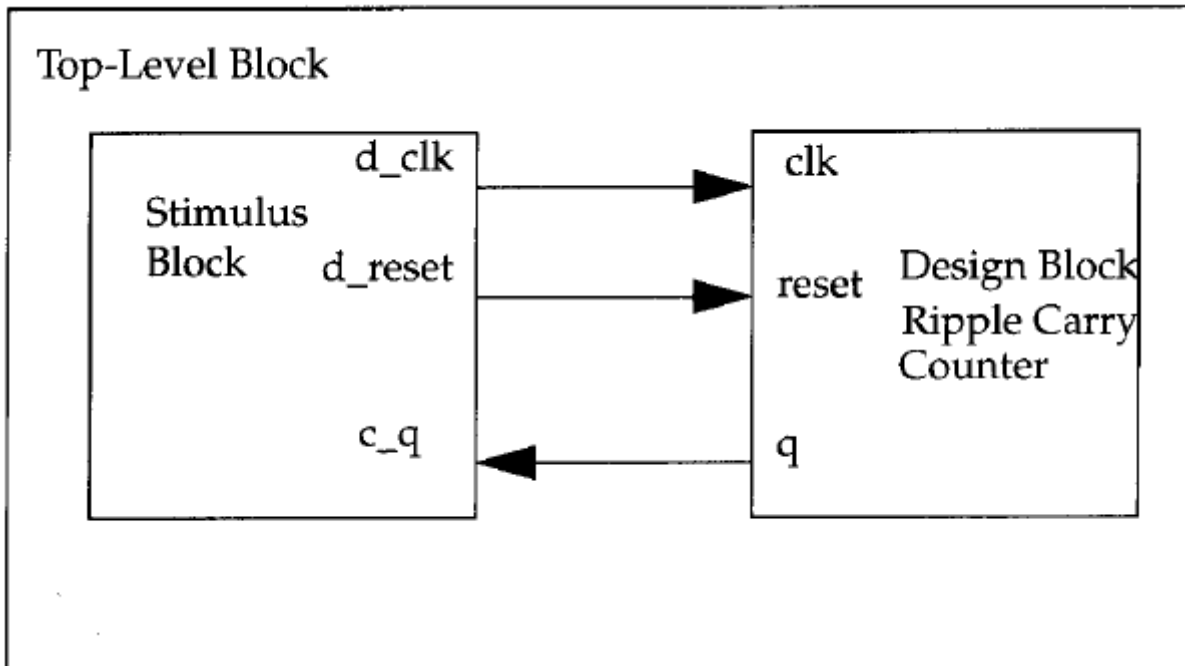


Figure 4: Separate Top-Level Module

- Unsized Numbers
- Special Values
- Additional Rules
- * Strings
- * Identifiers and Keywords
- * Escaped Identifiers
- 3. Data Types in Verilog
 - * Value Set
- 4. Nets
- 5. Registers
- 6. Vectors
 - * Vector Operations
- 7. Integer, Real, and Time Data Types
 - * Integer
 - * Real
 - * Time
- 8. Arrays
- 9. Memories
- 10. Parameters and Constants
 - * Local Parameters
- 11. Strings in Registers
- 12. System Tasks
 - * Displaying Information
 - * Monitoring Signals
 - * Simulation Control

- 13. Compiler Directives
 - * define
 - * include
-

4 Verilog Basic Concepts

4.1 1. Overview

This chapter establishes the foundational constructs and conventions used in Verilog HDL. These concepts are essential for understanding how Verilog models real hardware behavior, including data representation, simulation, and coding structure.

4.2 2. Lexical Conventions

Verilog syntax is similar to the C programming language and consists of tokens such as identifiers, keywords, numbers, operators, and strings.

4.2.1 General Rules

- Verilog is a case-sensitive language.
- Keywords are always written in lowercase.
- Code is interpreted as a stream of tokens.

4.2.2 Whitespace

- Spaces, tabs, and newlines are ignored except when separating tokens.
- Whitespace inside strings is preserved.

4.2.3 Comments

- Single-line comments start with `//`.
- Multi-line comments are enclosed within `/* */`.
- Nested multi-line comments are not allowed.

4.2.4 Operators

- Unary operators act on a single operand.
- Binary operators operate between two operands.
- Ternary operators use the conditional format `?:`.

4.2.5 Number Representation

- Numbers can be specified as sized or unsized.

4.2.5.1 Sized Numbers

- Format: `<size>'<base><value>`
- Supported bases: binary (b), decimal (d), hexadecimal (h), octal (o)
- Eg. `4'b1111`; `12'habc`;

4.2.5.2 Unsized Numbers

- Default to decimal and at least 32 bits.
- Eg. `'b1111`; `'hc3`; // Both of these are 32 bit numbers

4.2.5.3 Special Values

- x represents unknown values.
- z represents high impedance (floating state).
- ? can be used as an alternative to z in specific contexts.

4.2.5.4 Additional Rules

- Negative numbers use a minus sign before the size. Eg. `-6'd3`;
- Underscores improve readability and are ignored. Eg. `8'b1110_0111`;
- Automatic bit extension applies based on the most significant bit.

4.2.6 Strings

- Strings are enclosed in double quotes.
- They must be contained on a single line.
- Each character occupies one byte (ASCII).

4.2.7 Identifiers and Keywords

- Identifiers name variables, modules, and signals.
- They consist of letters, digits, underscores, and \$.
- They must not start with a digit or \$.

4.2.8 Escaped Identifiers

- Begin with `\` and end with whitespace.
- Allow use of special characters in names.

4.3 3. Data Types in Verilog

Verilog data types closely model hardware behavior and signal representation.

4.3.1 Value Set

Verilog supports four logic values: - 0 represents logic low. - 1 represents logic high. - x represents unknown. - z represents high impedance.

Signal strengths are also defined to resolve conflicts between multiple drivers.

4.4 4. Nets

- Nets represent physical connections between hardware elements.
- They continuously reflect the value driven by connected devices.
- The most commonly used net type is wire.
- Default value of a net is z if undriven.

Key characteristics: - Nets do not store values. - They must be driven by some source.

4.5 5. Registers

- Registers represent storage elements in Verilog.
- They retain their value until explicitly changed.
- Declared using the reg keyword.

Important distinctions: - A Verilog register is not necessarily a physical flip-flop. - Registers do not require a clock to update values. - Default value of a register is x.

Registers can also be declared as signed for arithmetic operations.

4.6 6. Vectors

- Vectors represent multi-bit nets or registers.
- Declared using range notation such as [msb:lsb].

Key concepts: - Default is scalar (1-bit) if no range is specified. - Bit ordering determines significance, with the left index as MSB.

4.6.1 Vector Operations

- Individual bits and ranges can be accessed using indexing.
 - Part-select allows selection of subsets of bits.
 - Variable part-select enables dynamic access using expressions.
-

4.7 7. Integer, Real, and Time Data Types

4.7.1 Integer

- Used for general-purpose computations.
- Typically at least 32 bits and signed.

4.7.2 Real

- Used for floating-point values.
- Supports decimal and scientific notation.
- Values are rounded when assigned to integers.

4.7.3 Time

- Used to store simulation time.
 - Typically at least 64 bits.
 - Retrieved using the system function \$time.
-

4.8 8. Arrays

- Arrays allow storage of multiple elements of the same type.
- Can be one-dimensional or multi-dimensional.

Key characteristics: - Each element can be a scalar or vector. - Accessed using index notation. - Arrays differ from vectors, which represent a single multi-bit value.

4.9 9. Memories

- Memories are modeled as one-dimensional arrays of registers.
- Used to represent RAM, ROM, and register files.

Key points: - Each element represents a word. - Each word can be multiple bits wide. - Access is performed using a single index.

4.10 10. Parameters and Constants

- Parameters define constants within a module.
- Declared using the parameter keyword.

Key advantages: - Enable configurable and reusable designs. - Can be overridden during module instantiation.

4.10.1 Local Parameters

- Declared using localparam.
 - Cannot be modified externally.
 - Useful for defining fixed internal constants such as state encodings.
-

4.11 11. Strings in Registers

- Strings can be stored in register variables.
- Each character occupies 8 bits.
- If the register is wider than the string, unused bits are zero-filled.
- If narrower, the string is truncated.

Special characters such as newline and tab are supported using escape sequences.

4.12 12. System Tasks

System tasks are built-in functions used for simulation control and debugging. They are identified by a \$ prefix.

4.12.1 Displaying Information

- \$display prints values, variables, or expressions.
- Supports formatted output similar to C's printf.
- Automatically appends a newline.

4.12.2 Monitoring Signals

- \$monitor continuously tracks specified signals.
- Outputs values whenever any monitored signal changes.
- Only one active monitor is allowed at a time.

4.12.3 Simulation Control

- \$stop pauses simulation for debugging.
 - \$finish terminates simulation execution.
-

4.13 13. Compiler Directives

Compiler directives control preprocessing behavior and are identified by a backtick (`).

4.13.1 define

- Used to create macros or constants.
- Improves readability and maintainability.

4.13.2 include

- Used to include contents of another file.
 - Commonly used for header files and shared definitions.
-

- Verilog Modules and Ports
 - 1. Overview of Modules
 - * Structure of a Module
 - 2. Ports and Module Interface
 - * Key Characteristics
 - 3. Port List
 - 4. Port Declarations
 - * Important Rules
 - * ANSI C Style Declaration
 - 5. Port Connection Rules
 - * Inputs
 - * Outputs

- * Inouts
 - * Width Matching
 - * Unconnected Ports
 - 6. Port Connection Methods
 - * Ordered List Connection
 - * Named Connection
 - 7. Hierarchical Name Referencing
 - * Key Concepts
 - * Benefits
 - * Special Usage
-

5 Verilog Modules and Ports

5.1 1. Overview of Modules

A module is the fundamental building block in Verilog and represents a self-contained unit of functionality. A module encapsulates internal implementation while exposing an interface through ports.

5.1.1 Structure of a Module

A Verilog module typically consists of the following components:

- Module declaration, which includes the keyword `module`, module name, and optional port list.
- Port declarations and optional parameters defined at the beginning.
- Internal components, which may include:
 - Variable declarations
 - Dataflow statements (`assign`)
 - Behavioral blocks (`initial`, `always`)
 - Instantiations of lower-level modules
 - Tasks and functions
- The module definition always ends with the keyword `endmodule`.

Only the module declaration and `endmodule` are mandatory. All other components are optional and can be arranged in any order within the module.

Verilog allows multiple modules to be defined within a single file, and their order of definition is not constrained.

5.2 2. Ports and Module Interface

Ports define the interface through which a module communicates with its external environment. They are analogous to input/output pins of a hardware component.

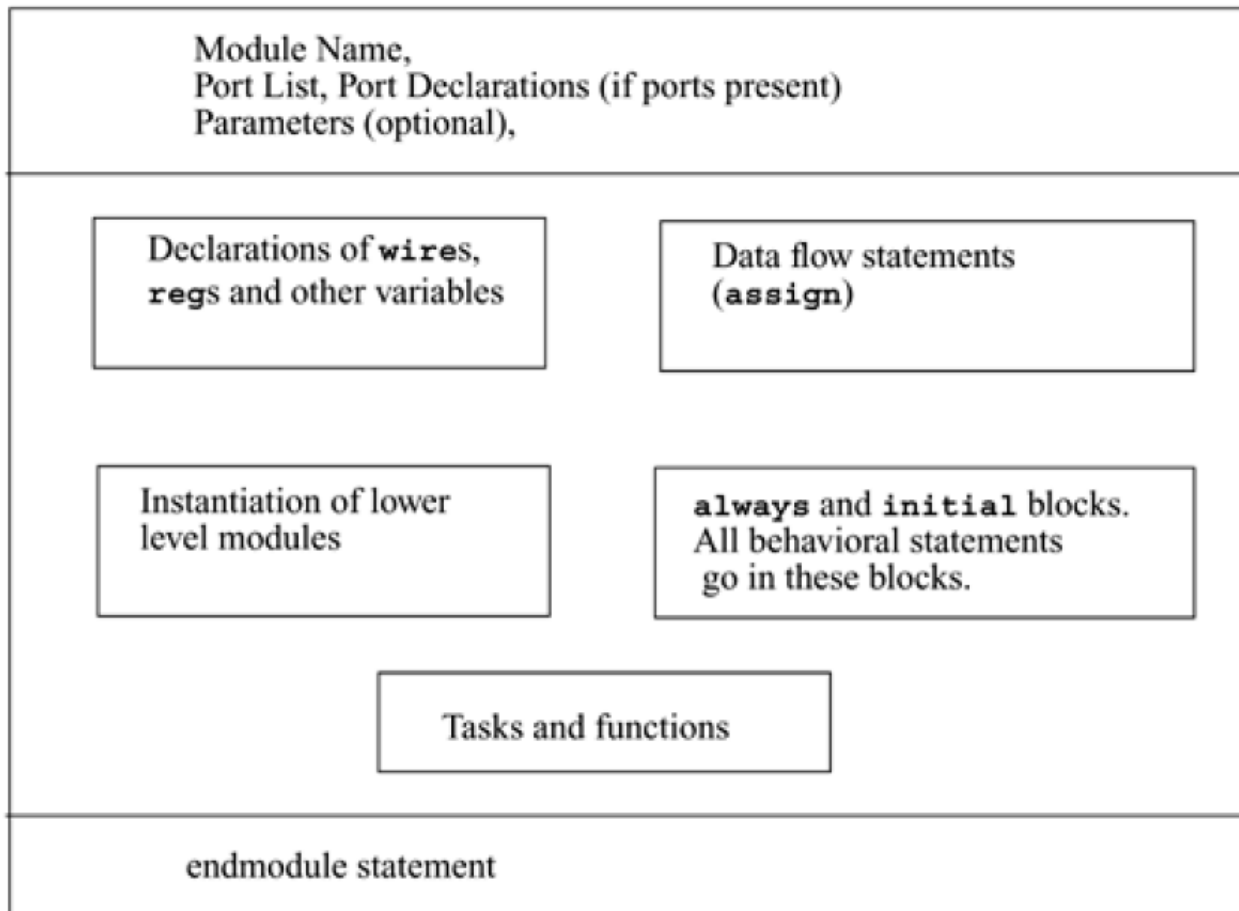


Figure 5: Structure of a verilog module

5.2.1 Key Characteristics

- Ports are the only visible interface of a module.
 - Internal implementation is hidden, enabling abstraction and modular design.
 - Changes to internal logic do not affect external connections as long as the port interface remains unchanged.
-

5.3 3. Port List

- A module may optionally include a list of ports.
- If a module does not interact with external signals, it may have no port list.
- A top-level module in simulation often has no ports because it acts as the root.

Example: - A functional module such as an adder includes ports. - A testbench or top-level module may not include ports.

5.4 4. Port Declarations

Each port in the port list must be declared with its direction:

- input defines incoming signals.
- output defines outgoing signals.
- inout defines bidirectional signals.

5.4.1 Important Rules

- Ports are implicitly of type wire unless specified otherwise.
- Input and inout ports must always be of type net (wire).
- Output ports can be of type wire or reg.

If an output needs to store its value across time (for example, in sequential logic), it must be declared as reg.

5.4.2 ANSI C Style Declaration

Verilog supports an alternative syntax where port direction and type are declared directly in the module header. This avoids duplication and improves readability.

5.5 5. Port Connection Rules

When connecting modules during instantiation, specific rules must be followed:

5.5.1 Inputs

- Internally, inputs must be nets.
- Externally, inputs can connect to either nets or registers.

5.5.2 Outputs

- Internally, outputs can be nets or registers.
- Externally, outputs must connect to nets and cannot connect directly to registers.

5.5.3 Inouts

- Internally and externally, inout ports must always be nets.

5.5.4 Width Matching

- Connections between ports of different widths are allowed but typically generate warnings.

5.5.5 Unconnected Ports

- Ports may be intentionally left unconnected.
- This is useful for debug signals or unused outputs.

Incorrect port connections, such as connecting an output port to a register externally, result in errors.

5.6 6. Port Connection Methods

Verilog provides two methods for connecting module ports to external signals during instantiation.

5.6.1 Ordered List Connection

- Signals are connected based on the order of ports in the module definition.
- The sequence of signals must exactly match the port list.

This method is simple but error-prone for large modules.

5.6.2 Named Connection

- Signals are connected explicitly using port names.
- The order of connections does not matter.
- Only required ports need to be specified.

This method is preferred for large designs due to improved readability and reduced errors.

5.7 7. Hierarchical Name Referencing

Verilog supports hierarchical design, where modules are instantiated within other modules. Each identifier in the design can be uniquely referenced using a hierarchical name.

5.7.1 Key Concepts

- The top-level module is called the root module.
- Each instance and signal has a unique position in the hierarchy.

- Hierarchical names are constructed using dot notation.

Example structure: - top_module.instance_name.signal_name

5.7.2 Benefits

- Allows access to signals across different levels of hierarchy.
- Enables debugging and monitoring of internal signals.

5.7.3 Special Usage

- The %m format specifier in \$display can be used to print the hierarchical name during simulation.

-
- Gate-Level Modeling in Verilog
 - 1. Overview of Gate-Level Modeling
 - 2. Gate Primitives in Verilog
 - * Categories of Gates
 - And/Or Type Gates
 - Buf/Not Type Gates
 - Controlled Gates (Bufif/Notif)
 - 3. Gate Instantiation
 - 4. Arrays of Gate Instances
 - 5. Gate Delays
 - * Types of Delays
 - Rise Delay
 - Fall Delay
 - Turn-Off Delay
 - 6. Delay Specification
 - 7. Min, Typical, and Max Delays
 - * Usage
 - 8. Timing Behavior and Simulation
-

6 Gate-Level Modeling in Verilog

6.1 1. Overview of Gate-Level Modeling

Gate-level modeling describes digital circuits using logic gates such as AND, OR, and NOT. It operates at a lower level of abstraction than RTL and provides a direct correspondence between circuit diagrams and Verilog code.

This modeling style is intuitive for designers familiar with digital logic because each Verilog construct maps closely to physical hardware components. While switch-level modeling is even lower, it is rarely used due to complexity.

6.2 2. Gate Primitives in Verilog

Verilog provides predefined gate primitives that can be instantiated directly without defining modules.

6.2.1 Categories of Gates

6.2.1.1 And/Or Type Gates

- These gates have one output and one or more inputs.
- The first terminal in the instantiation is always the output, followed by inputs.
- Output updates whenever any input changes.

Supported gates include: - and, or, xor - nand, nor, xnor

These gates can accept multiple inputs, and their outputs are computed by applying logic iteratively across all inputs.

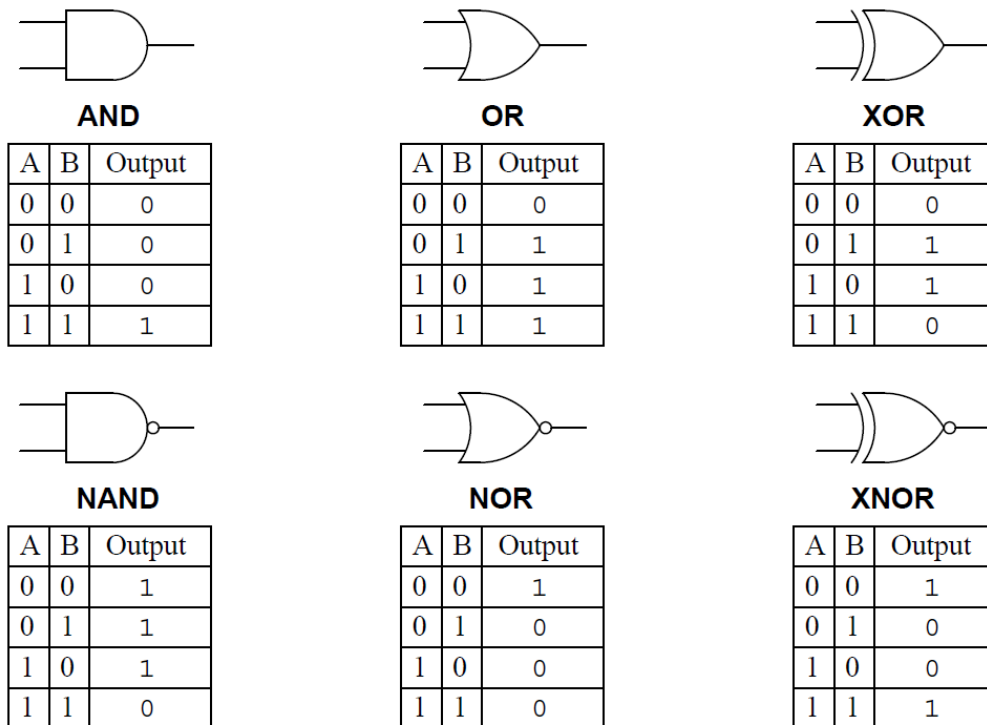


Figure 6: Logic Gates

6.2.1.2 Buf/Not Type Gates

- These gates have one input and one or more outputs.
- The last terminal represents the input, while preceding terminals are outputs.

Supported gates include: - buf, which passes the input unchanged. - not, which inverts the input.

6.2.1.3 Controlled Gates (Bufif/Notif)

- These gates include an additional control signal.

- They drive the output only when the control signal is active.
- When inactive, the output is high impedance (z).

Supported gates include: - bufif1, bufif0 - notif1, notif0

These gates are useful in designs where multiple drivers share a common signal line.

6.3 3. Gate Instantiation

- Gate primitives are instantiated similarly to modules.
- Instance names are optional for primitives, allowing concise descriptions.
- Multiple inputs or outputs can be specified directly in the instantiation.

Example concepts: - Gates can be used to directly translate logic diagrams into Verilog. - No separate module definition is required for primitive gates.

```
wire OUT, IN1, IN2;
```

```
// basic gate instantiations.
and a1(OUT, IN1, IN2);
nand na1(OUT, IN1, IN2);
or or1(OUT, IN1, IN2);
nor nor1(OUT, IN1, IN2);
xor x1(OUT, IN1, IN2);
xnor nx1(OUT, IN1, IN2);
buf b1(OUT1, IN);
not n1(OUT1, IN);
bufif1 b1 (out, in, ctrl);
bufif0 b0 (out, in, ctrl);
notif1 n1 (out, in, ctrl);
notif0 n0 (out, in, ctrl);

// More than two inputs; 3 input nand gate
nand na1_3inp(OUT, IN1, IN2, IN3);
buf b1_2out(OUT1, OUT2, IN);

// gate instantiation without instance name
and (OUT, IN1, IN2); // legal gate instantiation
not (OUT1, IN); // legal gate instantiation
```

6.4 4. Arrays of Gate Instances

Verilog allows creation of arrays of gate instances to simplify repetitive designs.

```
wire [3:0] OUT, IN1, IN2;
```

```
// basic gate instantiations.
nand n_gate[3:0](OUT, IN1, IN2);
```

```
// This is equivalent to the following 4 instantiations
nand n_gate0(OUT[0], IN1[0], IN2[0]);
nand n_gate1(OUT[1], IN1[1], IN2[1]);
nand n_gate2(OUT[2], IN1[2], IN2[2]);
nand n_gate3(OUT[3], IN1[3], IN2[3]);
```

- This is useful when performing identical operations across vectors.
- Each instance operates on corresponding bits of input and output vectors.

This approach significantly reduces code duplication and improves readability.

6.5 5. Gate Delays

Real hardware exhibits propagation delays, which can be modeled in Verilog.

6.5.1 Types of Delays

6.5.1.1 Rise Delay

- Represents the time taken for output to transition to logic 1.

6.5.1.2 Fall Delay

- Represents the time taken for output to transition to logic 0.

6.5.1.3 Turn-Off Delay

- Represents the time taken for output to transition to high impedance (z).

If the output transitions to an unknown value (x), the minimum of the specified delays is used.

6.6 6. Delay Specification

Verilog allows different levels of delay specification:

- A single delay value applies to all transitions.
- Two delay values specify rise and fall delays.
- Three delay values specify rise, fall, and turn-off delays explicitly.
- If no delay is specified, the default is zero.

This flexibility allows designers to model timing behavior with varying levels of detail.

6.7 7. Min, Typical, and Max Delays

Each delay type (rise, fall, turn-off) can include three values:

- Minimum delay represents the best-case scenario.

- Typical delay represents nominal behavior.
- Maximum delay represents worst-case conditions.

6.7.1 Usage

- One of these values is selected during simulation.
- Selection is typically controlled via simulator options.

This feature allows designers to analyze circuit behavior under different process variations without modifying the design.

```
// Delay of delay_time for all transitions
and #(delay_time) a1(out, i1, i2);

// Rise and Fall Delay Specification.
and #(rise_val, fall_val) a2(out, i1, i2);

// Rise, Fall, and Turn-off Delay Specification
and #(rise_val, fall_val, turnoff_val) a3(out, i1, i2);

// Three delays - min max & typical
// if +mindelays, rise= 2 fall= 3 turn-off = 4
// if +typdelays, rise= 3 fall= 4 turn-off = 5
// if +maxdelays, rise= 4 fall= 5 turn-off = 6
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1, i2);
```

6.8 8. Timing Behavior and Simulation

- Gate delays affect the timing of signal transitions in simulation.
- Outputs do not change immediately but follow specified delays.
- Intermediate signals also exhibit delays, impacting overall circuit timing.

Example insight: - A change in input propagates through gates sequentially, each adding its delay.
 - This results in observable timing differences between signals in waveforms.

-
- Dataflow Modeling in Verilog
 - 1. Overview of Dataflow Modeling
 - 2. Continuous Assignments
 - * Key Characteristics
 - 3. Implicit Continuous Assignment
 - * Key Point
 - 4. Implicit Net Declaration
 - 5. Delays in Continuous Assignments
 - * 5.1 Regular Assignment Delay
 - * 5.2 Implicit Assignment Delay
 - * 5.3 Net Declaration Delay
 - * Key Insight

- 6. Expressions, Operands, and Operators
 - * Expressions
 - * Operands
 - * Operators
 - 7. Operator Types in Verilog
 - * 7.1 Arithmetic Operators
 - * 7.2 Logical Operators
 - * 7.3 Relational Operators
 - * 7.4 Equality Operators
 - * 7.5 Bitwise Operators
 - * 7.6 Reduction Operators
 - * 7.7 Shift Operators
 - * 7.8 Concatenation Operator
 - * 7.9 Replication Operator
 - * 7.10 Conditional Operator
 - 8. Operator Precedence
-

7 Dataflow Modeling in Verilog

7.1 1. Overview of Dataflow Modeling

Dataflow modeling describes digital circuits in terms of how data moves and is transformed, rather than explicitly instantiating gates.

- It operates at a higher abstraction level compared to gate-level modeling.
 - It is widely used in modern design because logic synthesis tools can automatically convert dataflow descriptions into gate-level implementations.
 - In practice, designers combine dataflow and behavioral modeling to create RTL (Register Transfer Level) designs.
 - This approach improves productivity by allowing designers to focus on functionality and data movement instead of low-level hardware details.
-

7.2 2. Continuous Assignments

Continuous assignments are the fundamental construct in dataflow modeling and are used to drive values onto nets.

7.2.1 Key Characteristics

- Defined using the assign keyword.
- The left-hand side must always be a net (e.g., wire), not a register.
- The assignment is continuously active and updates whenever any operand on the right-hand side changes.
- The right-hand side can include nets, registers, constants, or function calls.
- Optional delays can be specified to model timing behavior.

Example

```
assign out = in1 & in2;
```

This continuously updates out whenever in1 or in2 changes.

7.3 3. Implicit Continuous Assignment

Verilog allows combining net declaration and assignment into a single statement.

Example

```
wire out = in1 & in2;
```

This is equivalent to declaring a wire and assigning it using a separate assign statement.

7.3.1 Key Point

- Only one implicit assignment is allowed per net because a net can be declared only once.
-

7.4 4. Implicit Net Declaration

If a signal appears on the left-hand side of an assignment and is not declared, Verilog implicitly declares it as a net.

Example

```
assign out = in1 & in2;
```

Here, out is automatically treated as a wire if not previously declared.

7.5 5. Delays in Continuous Assignments

Delays define when the assigned value appears on the output after input changes.

7.5.1 5.1 Regular Assignment Delay

- Delay is specified in the assign statement.

```
assign #10 out = in1 & in2;
```

- The output updates after 10 time units.
- Uses inertial delay, meaning short pulses (shorter than delay) are ignored.

7.5.2 5.2 Implicit Assignment Delay

- Delay is specified during net declaration.

```
wire #10 out = in1 & in2;
```

7.5.3 5.3 Net Declaration Delay

- Delay applies to any assignment to the net.

```
wire #10 out;  
assign out = in1 & in2;
```

7.5.4 Key Insight

- All three methods achieve similar timing behavior but differ in syntax and scope.
-

7.6 6. Expressions, Operands, and Operators

Dataflow modeling relies heavily on expressions.

7.6.1 Expressions

- Combinations of operands and operators that evaluate to a value.

Example:

```
out = a ^ b;
```

7.6.2 Operands

- Inputs to expressions, which can include:
 - Constants
 - Nets and registers
 - Bit-selects and part-selects
 - Function calls

7.6.3 Operators

- Symbols that perform operations on operands.
-

7.7 7. Operator Types in Verilog

Verilog provides a wide range of operators to model complex logic.

7.7.1 7.1 Arithmetic Operators

- Perform mathematical operations such as addition, subtraction, multiplication, division, modulus, and exponentiation.
- If any operand contains unknown (x), the result becomes unknown.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	*	Multiply	Two

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	/	Divide	Two
Arithmetic	+	Add	Two
Arithmetic	—	Subtract	Two
Arithmetic	%	Modulus	Two
Arithmetic	**	Power (Exponent)	Two

7.7.2 7.2 Logical Operators

- Include logical AND (&&), OR (||), and NOT (!).
- Always produce a 1-bit result (0, 1, or x).
- Any non-zero value is treated as true.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Logical	!	Logical Negation	One
Logical	&&	Logical AND	Two
Logical	\\	Logical OR	Two

7.7.3 7.3 Relational Operators

- Compare values using operators such as >, <, >=, <=.
- Result is 1 if true, 0 if false, and x if unknown values are involved.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Relational	>	Greater Than	Two
Relational	<	Less Than	Two
Relational	>=	Greater Than or Equal	Two
Relational	<=	Less Than or Equal	Two

7.7.4 7.4 Equality Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Equality	==	Equality	Two
Equality	!=	Inequality	Two
Equality	===	Case Equality	Two
Equality	!==	Case Inequality	Two

Key difference: * Logical equality returns x if operands contain unknowns. * Case equality compares exact bit patterns including x and z.

7.7.5 7.5 Bitwise Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Bitwise	~	Bitwise Negation	One
Bitwise	&	Bitwise AND	Two
Bitwise	\	Bitwise OR	Two
Bitwise	^	Bitwise XOR	Two
Bitwise	^^ or ~^	Bitwise XNOR	Two

7.7.6 7.6 Reduction Operators

- Operate on all bits of a single operand and produce a 1-bit result.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Reduction	&	Reduction AND	One
Reduction	~&	Reduction NAND	One
Reduction	\	Reduction OR	One
Reduction	~\	Reduction NOR	One
Reduction	^	Reduction XOR	One
Reduction	^^ or ~^	Reduction XNOR	One

Example:

```
parity = ^data; // XOR of all bits
```

7.7.7 7.7 Shift Operators

- Shift bits left or right.
- Arithmetic shifts preserve sign for signed data.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Shift	>>	Right Shift	Two
Shift	<<	Left Shift	Two
Shift	>>>	Arithmetic Right Shift	Two
Shift	<<<	Arithmetic Left Shift	Two

7.7.8 7.8 Concatenation Operator

- Combines multiple signals into a single vector.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Concatenation	{ }	Concatenation	Any Number

Example:

```
assign {carry, sum} = a + b;
```

7.7.9 7.9 Replication Operator

- Repeats a value multiple times.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Replication	{ { } }	Replication	Any Number

Example:

```
out = {4{1'b1}}; // 1111
```

7.7.10 7.10 Conditional Operator

- Acts like a multiplexer.
- If condition is unknown, both branches are evaluated and compared bitwise.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Conditional	?:	Conditional	Three

Example:

```
assign out = sel ? in1 : in0;
```

7.8 8. Operator Precedence

- Operators follow a defined precedence order, from unary operators (highest) to conditional operator (lowest).
- Parentheses should be used to avoid ambiguity and improve readability.

-
- Behavioral Modeling in Verilog
 - 1. Overview of Behavioral Modeling
 - 2. Structured Procedures: `initial` and `always`
 - * `initial` Block
 - * `always` Block
 - * Key Difference
 - 3. Procedural Assignments
 - * 3.1 Blocking Assignments (`=`)
 - * 3.2 Nonblocking Assignments (`<=`)
 - * Key Insight
 - 4. Timing Controls
 - * 4.1 Delay-Based Timing Control
 - Regular Delay
 - Intra-Assignment Delay
 - Zero Delay

- * 4.2 Event-Based Timing Control
 - Edge-Based
 - Named Events
 - Sensitivity List
 - Automatic Sensitivity
 - * 4.3 Level-Sensitive Timing Control
 - 5. Conditional Statements
 - * Types
 - Simple If
 - If-Else
 - Nested If-Else
 - * Key Behavior
 - 6. Case Statements (Multiway Branching)
 - * Basic Case
 - * Variants
 - casez
 - casex
 - 7. Looping Constructs
 - * 7.1 While Loop
 - * 7.2 For Loop
 - * 7.3 Repeat Loop
 - * 7.4 Forever Loop
 - 8. Sequential vs Parallel Blocks
 - * Sequential Block (begin—end)
 - * Parallel Block (fork—join)
 - * Key Insight
 - 9. Named Blocks and Control
 - * Named Blocks
 - * Disable Statement
 - 10. Generate Blocks (Elaboration-Time Constructs)
 - * Key Characteristics
 - * 10.1 Generate Loop
 - * 10.2 Generate Conditional
 - * 10.3 Generate Case
-

8 Behavioral Modeling in Verilog

8.1 1. Overview of Behavioral Modeling

Behavioral modeling describes a digital system in terms of its functionality and algorithm rather than its structural implementation.

- It operates at the highest level of abstraction in Verilog.
- Designers focus on algorithm behavior and system performance instead of gates or data paths.
- It is commonly used during architectural exploration before RTL implementation.
- The coding style resembles high-level programming languages such as C.

- Behavioral constructs provide flexibility to model complex control logic efficiently. :contentReferenceoaicite:0

8.2 2. Structured Procedures: **initial** and **always**

Behavioral code must be written inside structured procedural blocks.

8.2.1 **initial** Block

- Executes once, starting at simulation time 0, and then terminates.
- Multiple **initial** blocks run concurrently.
- Commonly used for:
 - Initialization
 - Testbench stimulus
 - Simulation control

Example:

```
initial begin
  a = 0;
  #10 b = 1;
end
```

8.2.2 **always** Block

- Executes continuously in a loop starting at time 0.
- Used to model ongoing hardware behavior such as clocks or sequential logic.

Example:

```
always #10 clk = ~clk;
```

8.2.3 Key Difference

- **initial** runs once, while **always** runs forever.
-

8.3 3. Procedural Assignments

Procedural assignments update variables such as reg, integer, or time.

8.3.1 3.1 Blocking Assignments (=)

- Executed sequentially within a block.
- Each statement completes before the next begins.
- Suitable for combinational logic modeling.

Example:

```
a = b;  
c = a; // uses updated value of a
```

8.3.2 3.2 Nonblocking Assignments (<=)

- Schedule updates without blocking subsequent statements.
- All right-hand side values are evaluated first, then updates occur.
- Used for modeling synchronous (clocked) logic.

Example:

```
a <= b;  
c <= a; // uses old value of a
```

8.3.3 Key Insight

- Blocking assignments are sequential.
 - Nonblocking assignments model parallel updates and avoid race conditions.
-

8.4 4. Timing Controls

Timing control defines when statements execute in simulation.

8.4.1 4.1 Delay-Based Timing Control

8.4.1.1 Regular Delay

- Delays execution of the entire statement.

```
#10 a = b;
```

8.4.1.2 Intra-Assignment Delay

- Evaluates RHS immediately but delays assignment.

```
a = #5 b;
```

8.4.1.3 Zero Delay

- Ensures execution occurs at the end of the current time step.
 - Used to manage race conditions but generally discouraged.
-

8.4.2 4.2 Event-Based Timing Control

Triggered by signal changes.

```
@(posedge clk) q = d;
```

8.4.2.2 Named Events

- Custom events can be triggered and detected.

```
always @(a or b or c)
```

```
always @(*)
```

- Automatically includes all signals used in the block.
 - Recommended for combinational logic.
-

8.4.3 4.3 Level-Sensitive Timing Control

- Waits for a condition to become true.

```
wait (enable) count = count + 1;
```

8.5 5. Conditional Statements

Used for decision-making in behavioral code.

8.5.1 Types

```
if (enable) out = in;
```

```

if ( a > b)
    max = a;
else
    max = b;

```

8.5.1.3 Nested If-Else

- Used for multiple conditions.

8.5.2 Key Behavior

- Condition evaluates to true (non-zero) or false (zero or unknown).
-

8.6 6. Case Statements (Multiway Branching)

Used when multiple conditions depend on a single expression.

8.6.1 Basic Case

```

case( sel )
    2'd0: out = a;
    2'd1: out = b;
    default: out = 0;
endcase

```

- Acts like a multiplexer.
- Only one branch executes.

8.6.2 Variants

8.6.2.1 casez

- Treats z as don't care.

8.6.2.2 casex

- Treats both x and z as don't care.
-

8.7 7. Looping Constructs

Loops allow repetitive execution inside procedural blocks.

8.7.1 7.1 While Loop

- Executes while condition is true.

```
while(i < 10)
    i = i + 1;
```

8.7.2 7.2 For Loop

- Includes initialization, condition, and increment.

```
for(i = 0; i < 10; i = i + 1)
    sum = sum + i;
```

8.7.3 7.3 Repeat Loop

- Executes fixed number of times.

```
repeat(8)
    data = data + 1;
```

8.7.4 7.4 Forever Loop

- Infinite loop, typically used with timing control.

```
forever #5 clk = ~clk;
```

8.8 8. Sequential vs Parallel Blocks

8.8.1 Sequential Block (begin-end)

- Statements execute in order.

```
begin
    a = 1;
    b = a;
end
```

8.8.2 Parallel Block (fork-join)

- Statements execute concurrently.

```

fork
  #5 a = 1;
  #10 b = 1;
join

```

8.8.3 Key Insight

- Sequential blocks ensure order.
 - Parallel blocks allow concurrency but may introduce race conditions.
-

8.9 9. Named Blocks and Control

8.9.1 Named Blocks

- Blocks can be named for hierarchy and control.

```

begin: block1

```

8.9.2 Disable Statement

- Terminates execution of a named block.

```

disable block1;

```

- Useful for breaking loops or early exits.
-

8.10 10. Generate Blocks (Elaboration-Time Constructs)

Generate constructs create hardware structures before simulation begins.

8.10.1 Key Characteristics

- Evaluated at compile/elaboration time.
 - Used for parameterized and scalable designs.
-

8.10.2 10.1 Generate Loop

- Repeats structures using `genvar`.

Example:

```

generate
  for (i=0; i<N; i=i+1)
    assign out[i] = a[i] ^ b[i];
endgenerate

```

8.10.3 10.2 Generate Conditional

- Instantiates different hardware based on parameters.

```
generate
  if (WIDTH < 8)
    small_unit u1 (...);
  else
    large_unit u2 (...);
endgenerate
```

8.10.4 10.3 Generate Case

- Selects implementation based on parameter values.
-

- Tasks and Functions in Verilog
 - 1. Overview
 - 2. Key Differences Between Tasks and Functions
 - * Functional Distinction
 - * Summary of Differences
 - 3. Tasks
 - * 3.1 When to Use Tasks
 - * 3.2 Task Declaration and Invocation
 - * Key Points
 - * 3.3 Task with Timing Control
 - * 3.4 Automatic (Re-entrant) Tasks
 - * Key Insight
 - 4. Functions
 - * 4.1 When to Use Functions
 - * 4.2 Function Declaration and Return Mechanism
 - * Key Insight
 - * 4.3 Function with Explicit Width
 - * 4.4 Automatic (Recursive) Functions
 - * Key Insight
 - 5. Constant Functions
 - 6. Signed Functions
 - 7. Best Practices
-

9 Tasks and Functions in Verilog

9.1 1. Overview

Tasks and functions are used to modularize and reuse commonly used behavioral code in Verilog designs.

- They help reduce code duplication and improve readability and maintainability.
- Both are defined within a module and are part of the design hierarchy.
- They are invoked from `initial`, `always`, or other tasks/functions.
- They contain only behavioral statements and cannot include `always` or `initial` blocks. :contentReferenceoaicite:0

9.2 2. Key Differences Between Tasks and Functions

9.2.1 Functional Distinction

- A **function** is used for combinational logic that:
 - Executes in zero simulation time.
 - Returns exactly one value.
 - Has only input arguments.
- A **task** is used for more general procedures that:
 - May include delays, events, or timing control.
 - Can return multiple values via output/inout arguments.
 - Can have zero or more arguments.

9.2.2 Summary of Differences

Feature	Function	Task
Execution time	Zero time	May consume time
Timing control	Not allowed	Allowed
Return value	Single value	No direct return, multiple outputs allowed
Arguments	Only inputs	Input, output, inout
Invocation capability	Can call functions only	Can call both tasks and functions

9.3 3. Tasks

9.3.1 3.1 When to Use Tasks

A task must be used when: - The logic includes delays or timing control. - Multiple outputs are required. - There are no input arguments.

9.3.2 3.2 Task Declaration and Invocation

Tasks are declared using `task` and `endtask`.

Example:

```
task bitwise_op;
  input [7:0] a, b;
```

```

    output [7:0] out;
begin
    out = a & b;
end
endtask

```

Invocation:

```
bitwise_op(A, B, OUT);
```

9.3.3 Key Points

- Arguments are passed in positional order.
 - Output values are returned when the task completes.
 - Tasks can call other tasks and functions.
-

9.3.4 3.3 Task with Timing Control

Tasks can include delays, making them suitable for sequential behavior.

Example:

```

task delayed_and;
    input a, b;
    output out;
begin
    #10 out = a & b;
end
endtask

```

9.3.5 3.4 Automatic (Re-entrant) Tasks

- By default, tasks are **static**, meaning variables are shared across calls.
- This can cause incorrect behavior if tasks are invoked concurrently.

To solve this, use automatic:

```
task automatic compute;
```

9.3.6 Key Insight

- Each call gets its own independent variable space.
 - Recommended when tasks may be called concurrently.
-

9.4 4. Functions

9.4.1 4.1 When to Use Functions

A function must satisfy all of the following:

- No delay or timing control.
 - At least one input argument.
 - Returns exactly one value.
 - No output or inout arguments.
 - No nonblocking assignments.
-

9.4.2 4.2 Function Declaration and Return Mechanism

Functions use an implicit variable (same name as function) to return values.

Example:

```
function parity;  
    input [7:0] data;  
    begin  
        parity = ^data;  
    end  
endfunction
```

Invocation:

```
p = parity(data);
```

9.4.3 Key Insight

- Return value is assigned to the function name.
 - Default return width is 1 bit unless specified.
-

9.4.4 4.3 Function with Explicit Width

Example:

```
function [31:0] shift;  
    input [31:0] data;  
    input dir;  
    begin  
        shift = (dir) ? (data >> 1) : (data << 1);  
    end  
endfunction
```

9.4.5 4.4 Automatic (Recursive) Functions

- Functions can be declared automatic for recursion or concurrent calls.

Example:

```
function automatic integer factorial;  
  input integer n;  
  begin  
    if (n <= 1)  
      factorial = 1;  
    else  
      factorial = n * factorial(n-1);  
    end  
endfunction
```

9.4.6 Key Insight

- Each call gets independent storage.
 - Enables recursion and safe concurrent usage.
-

9.5 5. Constant Functions

- Used to compute values at compile/elaboration time.
- Commonly used for parameter calculations.

Example:

```
function integer clog2;  
  input integer depth;  
  begin  
    for(clog2 = 0; depth > 0; clog2 = clog2 + 1)  
      depth = depth >> 1;  
    end  
endfunction
```

Usage:

```
input [clog2(256)-1:0] addr;
```

9.6 6. Signed Functions

- Functions can return signed values using signed.

Example:

```
function signed [31:0] compute;
```

- Enables signed arithmetic comparisons and operations.
-

9.7 7. Best Practices

- Use **functions** for combinational logic and calculations.
 - Use **tasks** for:
 - Sequential behavior
 - Delays and timing control
 - Multiple outputs
 - Avoid using static tasks when concurrent execution is possible; prefer automatic.
 - Keep functions simple and side-effect free.
 - Ensure correct argument ordering during task invocation.
 - Use functions for reusable logic like parity, encoding, or arithmetic operations.
-